TP N3 — Le modèle de langue et sa mesure

Module NLP, séance 3 — projet **ILIACode**

Travail Individuel

Pr. Asmaa RASSIL

ENSA Fès, Filière ILIA (3e année)

Le code couleur du document

Chaque nature d'objet porte sa couleur, la même d'un bout à l'autre du TP.

- **`bigramme.py`** est un fichier ou un dossier.
- **`compter`** est une des fonctions à compléter.
- **`pytest`** est une bibliothèque ou un objet importé.
- **slide 8** renvoie au cours de la séance.
- **notion importante**, **erreur ou test en échec**, **test au vert**.
- `commande tapée` ou affichage de l'écran, sans nature particulière.

Les parties du TP

1. Partie 0 — La mise en place
2. Partie 1 — Compter
3. Partie 2 — Prédire
4. Partie 3 — Lisser
5. Partie 4 — Mesurer
6. Partie 5 — Mesurer le dépôt
7. Partie 6 — Assainir
8. Partie 7 — Publier

Partie 0 — La mise en place

Lien avec le cours

Cette partie suit les **slides 2** (Le moteur prédit le token suivant) et **3** (Le déroulement de la séance). Elle prépare les fichiers du tableau du **slide 3**.

Étapes 0.1 et 0.2 — Le dossier de la séance

Étape 0.1. Placer le dossier de la séance dans votre dépôt.

Faites un clic droit sur **`seance3_etudiants.zip`**, choisissez *Extraire tout*, et placez le dossier **`seance3`** obtenu **à l'intérieur** de votre dossier **`iliacode`**, à côté de **`seance2`**.

Étape 0.2. Ouvrir le dépôt dans VS Code.

Lancez VS Code, menu *Fichier*, *Ouvrir le dossier*, et choisissez **`iliacode`**. Dans la colonne de gauche, dépliez **`seance3`**. Vous devez y voir **`bigramme.py`**, **`perplexite.py`**, **`hygiene.py`**, **`test_seance3.py`** et **`mesure_perplexite.py`**.

Étape 0.2 — La brique de la séance 2

Cette séance **réutilise** votre travail de la séance 2. Dépliez aussi **seance2** et vérifiez qu'il contient bien ces trois éléments.

- **bpe.py**, votre tokeniseur, avec ses 10 tests au vert.
- **merges.json**, la liste des fusions créée à la fin du TP N2.
- **donnees**, avec ses dossiers **entrainement** et **test**.

En cas de problème

Si **merges.json** manque, ouvrez un terminal dans **seance2** et relancez `python mesure_fertilite.py`, le fichier est recréé en quelques secondes.

Étape 0.3 — Le terminal

Étape 0.3. Ouvrir le terminal et entrer dans le dossier.

Menu *Terminal*, *Nouveau terminal*. Activez votre environnement virtuel.

```
.venv\Scripts\activate.bat
```

Son nom doit apparaître entre parenthèses au début de la ligne. Entrez ensuite dans le dossier de la séance.

```
cd seance3
```

Aucune bibliothèque nouvelle n'est à installer.

Étape 0.4 — Les 14 tests une première fois

Étape 0.4. Lancer les 14 tests une première fois.

```
pytest -v
```

Vérification avant de continuer

Quatorze lignes se terminent par **FAILED**, et la dernière ligne de l'écran est la suivante (le temps affiché varie).

```
============================== 14 failed in 0.09s ==============================
```

C'est **normal**. Les fonctions sont encore vides. La séance consiste à faire passer ces 14 lignes au vert.

Étape 0.4 — Les trois fichiers à compléter

Chaque fichier porte une brique de la séance, comme le **slide 3** l'annonce.

- **`bigramme.py`** est le premier moteur, il compte puis il prédit.
- **`perplexite.py`** est l'instrument de mesure, il ne sait rien du moteur mesuré.
- **`hygiene.py`** retire les secrets d'un texte avant tout comptage.

Chaque fonction à écrire se termine par une ligne `raise NotImplementedError("partie N du TP, à compléter")`. Compléter une fonction, c'est **remplacer cette ligne** par le vrai code.

Étape 0.4 — La règle du parcours

Deux constantes en haut de **`bigramme.py`** servent partout.

- `DEBUT`, le token de bord `<s>`, placé avant chaque ligne.
- `FIN_SEQ`, le token de bord `</s>`, placé après chaque ligne.

Règle du parcours

Après chaque modification d'un fichier, enregistrez avec **Ctrl+S** avant de lancer les tests. Dans les cadres gris qui montrent l'interpréteur, ne tapez pas les `>>>`, tapez seulement ce qui vient après. Après chaque modification, **quittez l'interpréteur** avec `exit()` et relancez `python`, sinon il garde l'ancienne version du fichier.

Les parties du TP

1. Partie 0 — La mise en place
2. Partie 1 — Compter
3. Partie 2 — Prédire
4. Partie 3 — Lisser
5. Partie 4 — Mesurer
6. Partie 5 — Mesurer le dépôt
7. Partie 6 — Assainir
8. Partie 7 — Publier

Partie 1 — Compter

Lien avec le cours

Cette partie applique le **slide 8** (Les deux phases du bigramme). L'étape 1.1 fabrique à la main les couples de voisins d'une ligne, l'étape 1.3 écrit la phase 1, et son essai retrouve le tableau des comptes du **slide 8**.

Étape 1.1 — Les couples de voisins d'une ligne

Étape 1.1. Voir ce que le modèle compte (slide 8).

Ouvrez l'interpréteur avec la commande `python`, puis tapez ces lignes.

```
>>> ligne = ["for", "ticket", "in", "tickets"]
>>> bordee = ["<s>"] + ligne + ["</s>"]
>>> bordee
['<s>', 'for', 'ticket', 'in', 'tickets', '</s>']
>>> list(zip(bordee, bordee[1:]))
[('<s>', 'for'), ('for', 'ticket'), ('ticket', 'in'),
 ('in', 'tickets'), ('tickets', '</s>')]
>>> exit()
```

Étape 1.1 — La lecture des couples

- `bordee[1:]` est la même liste privée de son premier élément. **`zip`** met les deux listes côte à côte, chaque token se retrouve donc **à côté de son successeur**.
- Chaque couple se lit (contexte, token suivant). Une ligne de quatre mots donne **cinq couples**, car le token de fin `</s>` est prédit lui aussi.
- `<s>` n'apparaît qu'à gauche, il sert de contexte au premier mot et n'est **jamais prédit**.

Étape 1.2 — La fonction encadrer

Étape 1.2. Compléter la fonction encadrer (slide 6, Définition des tokens de bord).

Dans **bigramme.py**, trouvez **encadrer**. Remplacez sa ligne `raise` par la ligne suivante.

```
    return [DEBUT] + list(tokens) + [FIN_SEQ]
```

C'est le geste de l'étape 1.1, écrit une fois pour toutes. Enregistrez avec **Ctrl+S**.

Étape 1.3 — La fonction `compter`

Étape 1.3. Compléter la fonction `compter`, la phase 1 (slide 8).

Dans **`compter`**, remplacez la ligne `raise` par ces cinq lignes. Les lignes `paires, contextes = Counter(), Counter()` au-dessus et `return paires, contextes` en dessous restent en place.

```
    for tokens in sequences:
        bordee = encadrer(tokens)
        for v, w in zip(bordee, bordee[1:]):
            paires[(v, w)] += 1
            contextes[v] += 1
```

Enregistrez avec **Ctrl+S**.

Étape 1.3 — Ce que font les cinq lignes

- La première boucle parcourt les lignes du corpus, chacune étant une liste de tokens.
- **`encadrer`** ajoute les deux tokens de bord.
- La seconde boucle parcourt les couples de voisins, `v` est le contexte et `w` le token qui le suit.
- `paires[(v, w)] += 1` est le compte $C(v, w)$ du **slide 8**, et `contextes[v] += 1` est le compte $C(v)$.

Les deux **`Counter`** sont **l'artefact figé**. Un **`Counter`** répond 0 pour une clé qu'il n'a jamais vue, sans erreur.

Étape 1.3 — L'essai sur le mini-corpus

Le mini-corpus du **slide 8** est déjà écrit dans **`test_seance3.py`**, sous le nom `MINI`. Relancez `python`.

```
>>> from test_seance3 import MINI
>>> from bigramme import compter
>>> paires, contextes = compter(MINI)
>>> contextes["<s>"], contextes["in"]
(5, 4)
>>> paires[("in", "users")], paires[("in", "queue")]
(2, 1)
>>> paires[("ticket", "queue")]
0
>>> exit()
```

Ce sont les lignes <s> et `in` du tableau du **slide 8**. Le dernier compte vaut **0**, `ticket` n'a jamais été suivi de queue.

Étape 1.4 — Les deux tests de la partie

Étape 1.4. Lancer les deux tests de la partie.

```
pytest -v -k p1
```

Vérification avant de continuer

```
test_seance3.py::test_p1_encadrer PASSED                                 [ 50%]
test_seance3.py::test_p1_compter_la_table PASSED                         [100%]
====================== 2 passed, 12 deselected in 0.02s ======================
```

Les deux lignes se terminent par **PASSED**. **La phase 1 est écrite.**

Les parties du TP

1. Partie 0 — La mise en place
2. Partie 1 — Compter
3. Partie 2 — Prédire
4. Partie 3 — Lisser
5. Partie 4 — Mesurer
6. Partie 5 — Mesurer le dépôt
7. Partie 6 — Assainir
8. Partie 7 — Publier

Partie 2 — Prédire

Lien avec le cours

Cette partie applique les **slides 8** (la formule encadrée de la phase 2) et **9** (La première prédiction). L'étape 2.1 écrit la division, l'étape 2.2 le classement des candidats, l'étape 2.3 retrouve le tableau et les deux probabilités de ligne du **slide 9**.

Étape 2.1 — La fonction proba

Étape 2.1. Compléter la fonction proba, la phase 2 (slide 8, formule encadrée).

Remplacez la ligne `raise` de **proba** par ces trois lignes.

```
    if contextes[v] == 0:
        return 0.0
    return paires[(v, w)] / contextes[v]
```

La dernière ligne est la formule du **slide 8**, $C(v, w)$ divisé par $C(v)$. Les deux premières évitent une division par zéro quand le contexte v n'a jamais été vu. **Rien n'est recompté**, la fonction lit la table. Enregistrez.

Étape 2.2 — La fonction `predire`

Étape 2.2. Compléter la fonction `predire` (slide 9, tableau des candidats).

Remplacez la ligne `raise` de **`predire`** par ces quatre lignes.

```
    candidats = [(w, proba(v, w, paires, contextes))
                 for (contexte, w) in paires if contexte == v]
    candidats.sort(key=lambda couple: (-couple[1], couple[0]))
    return candidats[:combien]
```

Enregistrez avec **Ctrl+S**.

Étape 2.2 — Ce que font les quatre lignes

- Les deux premières parcourent toutes les paires de la table et ne gardent que celles dont le contexte est `v`. Chaque successeur `w` est rangé avec sa probabilité.
- `sort` trie les candidats. Le signe moins devant `couple[1]` place **la plus grande probabilité en premier**, et à probabilité égale l'ordre alphabétique départage.
- `[:combien]` ne garde que les premiers, cinq par défaut, ce sont les candidats entre lesquels le copilote choisit le token suivant de sa réponse.

Étape 2.3 — La première prédiction

Étape 2.3. Retrouver les chiffres du cours (slide 9).

Relancez `python`.

```
>>> from test_seance3 import MINI
>>> from bigramme import compter, proba, predire
>>> paires, contextes = compter(MINI)
>>> proba("for", "ticket", paires, contextes)
0.75
>>> predire("in", paires, contextes)
[('users', 0.5), ('queue', 0.25), ('tickets', 0.25)]
```

C'est le tableau du **slide 9**. Quand la réponse du copilote en est à `for ticket in`, le bigramme ne regarde que `in` et place **users** en premier, alors que la demande parle des tickets. Restez dans l'interpréteur.

Étape 2.3 — La probabilité d'une ligne entière

La fonction **`probas_du_texte`**, déjà écrite, rend la probabilité donnée à chaque token d'une ligne, le token de fin compris. Le dernier argument, 0, demande le comptage brut. **`math.prod`** multiplie ces facteurs.

```
>>> import math
>>> from bigramme import probas_du_texte
>>> vue = [["for", "ticket", "in", "tickets"]]
>>> probas_du_texte(vue, paires, contextes, 11, 0)
[0.8, 0.75, 0.75, 0.25, 1.0]
>>> math.prod(probas_du_texte(vue, paires, contextes, 11, 0))
0.11250000000000002
>>> nouvelle = [["for", "user", "in", "queue"]]
>>> math.prod(probas_du_texte(nouvelle, paires, contextes, 11, 0))
0.05
>>> exit()
```

Étape 2.3 — La lecture des deux nombres

- Les cinq facteurs de la première ligne sont ceux du **slide 9**, $4/5$, $3/4$, $3/4$, $1/4$ et $1/1$. Leur produit vaut **0,1125**, la machine affiche `0.11250000000000002` à cause de l'arrondi des nombres à virgule.
- La seconde ligne, `for user in queue`, **n'est écrite nulle part** dans le mini-corpus. Le modèle lui donne pourtant **0,05**, car chacun de ses couples a été vu séparément.

C'est la règle de la chaîne du **slide 5** au travail, la probabilité d'une ligne est le produit des probabilités de ses tokens.

Étape 2.4 — Les trois tests de la partie

Étape 2.4. Lancer les trois tests de la partie.

```
pytest -v -k p2
```

Vérification avant de continuer

```
test_seance3.py::test_p2_proba_lue_dans_la_table PASSED                  [ 33%]
test_seance3.py::test_p2_predire_apres_in PASSED                         [ 66%]
test_seance3.py::test_p2_proba_d_une_ligne PASSED                        [100%]
====================== 3 passed, 11 deselected in 0.02s ======================
```

Lancez ensuite `pytest` sans option. La dernière ligne doit annoncer **9 failed**, **5 passed**.

Partie 2 — Dans une autre application

Dans une autre application

Type d'application. Un assistant qui répond aux demandes des médecins d'un hôpital, ou des conseillers d'une banque.

Comment. Les deux fonctions `compter` et `predire` ne savent rien du code informatique. Données aux comptes rendus de l'hôpital à la place de `MINI`, elles classent les candidats pour le terme suivant de la réponse, **appris sur les seuls documents de l'établissement**, sans qu'aucun texte n'en sorte.

Les parties du TP

1. Partie 0 — La mise en place
2. Partie 1 — Compter
3. Partie 2 — Prédire
4. Partie 3 — Lisser
5. Partie 4 — Mesurer
6. Partie 5 — Mesurer le dépôt
7. Partie 6 — Assainir
8. Partie 7 — Publier

Partie 3 — Lisser

Lien avec le cours

Cette partie applique le **slide 10** (Le zéro du jamais vu et le lissage additif). L'étape 3.1 rejoue le produit nul du cadre Problème, l'étape 3.2 écrit la formule encadrée de la Définition et retrouve le $1/12$ du cadre Vérification.

Étape 3.1 — Le zéro du jamais vu

Étape 3.1. Voir un seul zéro détruire le produit (slide 10, Problème).

Relancez python.

```
>>> import math
>>> from test_seance3 import MINI
>>> from bigramme import compter, probas_du_texte
>>> paires, contextes = compter(MINI)
>>> cassee = [["if", "user", "is", "None"]]
>>> probas_du_texte(cassee, paires, contextes, 11, 0)
[0.2, 0.0, 0.0, 1.0, 1.0]
>>> math.prod(probas_du_texte(cassee, paires, contextes, 11, 0))
0.0
>>> exit()
```

La ligne est correcte et tous ses mots sont connus. Mais `if` n'a jamais été suivi de `user`, ni `user` de `is`, et le produit tombe à **0**.

Étape 3.2 — La fonction proba_lissee

Étape 3.2. Compléter la fonction proba_lissee (slide 10, Définition).

Remplacez la ligne `raise` de **proba_lissee** par la ligne suivante.

```
    return (paires[(v, w)] + k) / (contextes[v] + k * taille_V)
```

C'est la formule encadrée du **slide 10**. Le numérateur ajoute k au compte, même nul. Le dénominateur ajoute k autant de fois qu'il y a de tokens prédictibles, `taille_V`, pour que les probabilités somment toujours à 1. Enregistrez avec **Ctrl+S**.

Étape 3.2 — L'essai

Relancez `python`. Le mini-corpus compte dix mots distincts et le token de fin, donc `taille_V` vaut 11.

```
>>> import math
>>> from test_seance3 import MINI
>>> from bigramme import compter, proba_lissee, probas_du_texte
>>> paires, contextes = compter(MINI)
>>> proba_lissee("if", "user", paires, contextes, 11, 1)
0.08333333333333333
>>> cassee = [["if", "user", "is", "None"]]
>>> math.prod(probas_du_texte(cassee, paires, contextes, 11, 1))
2.4112654320987653e-05
>>> exit()
```

`0.0833...` est le $1/12$ du **slide 10**. Avec $k = 1$ le produit n'est plus nul, il vaut environ $0{,}000024$. La ligne est redevenue **mesurable**.

Étape 3.3 — Les deux tests de la partie

Étape 3.3. Lancer les deux tests de la partie.

```
pytest -v -k p3
```

Vérification avant de continuer

```
test_seance3.py::test_p3_lissage_supprime_le_zero PASSED                 [ 50%]
test_seance3.py::test_p3_lissage_reste_une_probabilite PASSED            [100%]
====================== 2 passed, 12 deselected in 0.02s ======================
```

Lancez ensuite `pytest` sans option. La dernière ligne doit annoncer **7 failed**, **7 passed**. **Le premier moteur est complet.**

— Pause de 15 minutes —

Les parties du TP

1. Partie 0 — La mise en place
2. Partie 1 — Compter
3. Partie 2 — Prédire
4. Partie 3 — Lisser
5. Partie 4 — Mesurer
6. Partie 5 — Mesurer le dépôt
7. Partie 6 — Assainir
8. Partie 7 — Publier

Partie 4 — Mesurer

Lien avec le cours

Cette partie applique les **slides 12** (La surprise d'un token) et **13** (L'entropie croisée et la perplexité). L'étape 4.1 écrit la Définition du **slide 12** et retrouve son tableau, les étapes 4.2 et 4.3 écrivent les deux Définitions du **slide 13**, l'étape 4.4 retrouve $1{,}75$ bit et $3{,}36$.

Étape 4.1 — La fonction surprise

Étape 4.1. Compléter la fonction surprise (slide 12, Définition).

Ouvrez **perplexite.py**. Remplacez la ligne `raise` de **surprise** par ces trois lignes.

```
    if p == 0:
        return math.inf
    return math.log2(1 / p)
```

`math.log2(1 / p)` est le $-\log_2 p$ du **slide 12**, écrit sans signe moins. Les deux premières lignes traitent le cas $p = 0$, où le logarithme n'existe pas. La surprise est alors **infinie**, et **math.inf** est la façon de l'écrire. Enregistrez.

Étape 4.1 — L'essai

Relancez `python`.

```
>>> from perplexite import surprise
>>> surprise(1), surprise(0.5), surprise(0.25), surprise(0.125)
(0.0, 1.0, 2.0, 3.0)
>>> surprise(0)
inf
>>> exit()
```

C'est le tableau du **slide 12**. Un token attendu avec certitude ne surprend pas, et chaque division de la probabilité par deux ajoute **un bit**, une question à réponse oui ou non.

Étapes 4.2 et 4.3 — La moyenne, puis la perplexité

Étape 4.2. Compléter la fonction `entropie_croisee` (slide 13, première Définition).

```
    return sum(surprise(p) for p in probas) / len(probas)
```

La somme des surprises divisée par le nombre de tokens, c'est la **surprise moyenne**.

Étape 4.3. Compléter la fonction `perplexite` (slide 13, seconde Définition).

```
    return 2 ** entropie_croisee(probas)
```

`2 ** h` se lit « 2 puissance h ». Enregistrez avec **Ctrl+S**.

Étape 4.4 — L'exemple du cours

Étape 4.4. Retrouver les chiffres du cours (slide 13).

Relancez `python`. Les quatre nombres sont les probabilités données aux tokens de `import numpy as np` dans le tableau du **slide 13**.

```
>>> from perplexite import entropie_croisee, perplexite
>>> probas = [0.5, 0.25, 0.5, 0.125]
>>> entropie_croisee(probas)
1.75
>>> perplexite(probas)
3.363585661014858
>>> perplexite([1 / 8] * 20)
8.0
>>> perplexite([0.5, 0, 0.5])
inf
>>> exit()
```

Étape 4.4 — La lecture des quatre résultats

- **1,75 bit** est la moyenne des surprises 1, 2, 1 et 3, et $2^{1,75} \approx$ **3,36** est la perplexité, les deux nombres rouges du **slide 13**.
- Un modèle qui donne $1/8$ à chacun de vingt tokens a une perplexité de **8**. La perplexité est bien **le nombre de candidats entre lesquels le modèle hésite**.
- **Un seul zéro** parmi les probabilités rend la perplexité infinie. Un modèle non lissé est donc **disqualifié** dès qu'il rencontre un couple nouveau.

Étape 4.5 — Les quatre tests de la partie

Étape 4.5. Lancer les quatre tests de la partie.

```
pytest -v -k p4
```

Vérification avant de continuer

```
test_seance3.py::test_p4_surprise PASSED                                 [ 25%]
test_seance3.py::test_p4_entropie_et_perplexite_de_l_exemple PASSED      [ 50%]
test_seance3.py::test_p4_perplexite_d_un_modele_uniforme PASSED          [ 75%]
test_seance3.py::test_p4_un_zero_rend_la_perplexite_infinie PASSED       [100%]
====================== 4 passed, 10 deselected in 0.03s ======================
```

L'instrument de mesure est écrit. Il resservira tel quel pour tous les moteurs des séances suivantes.

Les parties du TP

1. Partie 0 — La mise en place
2. Partie 1 — Compter
3. Partie 2 — Prédire
4. Partie 3 — Lisser
5. Partie 4 — Mesurer
6. Partie 5 — Mesurer le dépôt
7. Partie 6 — Assainir
8. Partie 7 — Publier

Étape 5.1 — La mesure sur le dépôt

Étape 5.1. Lancer la mesure sur le corpus réel (slide 14).

Le script **mesure_perplexite.py** va chercher dans **seance2** votre **bpe.py**, votre **merges.json** et le dossier **donnees**. Il transforme chaque ligne en tokens, puis applique le protocole du **slide 14**.

```
python mesure_perplexite.py
```

- Le bloc **train**, neuf lignes sur dix de **donnees/entrainement**, sert à compter.
- Le bloc **dev**, une ligne sur dix, sert à choisir k.
- Le bloc **test**, **donnees/test**, est lu une seule fois, à la fin.

Étape 5.1 — L'allure de l'écran

Vérification avant de continuer

```
hygiène pas encore écrite (partie 6), les lignes sont comptées telles quelles
train : 10048 lignes, 136659 tokens prédits, |V| = 2001
bigrammes distincts vus : 29448 sur 4004001 possibles, soit 0.74 %

comptage brut   train     26.5   dev      inf

k             train       dev
1             158.8     182.3
0.1            54.8      76.6
0.01           31.4      54.7
0.001          27.1      59.1

k retenu sur le dev : 0.01
```

Étape 5.1 — La lecture du tableau de k

L'écran précédent vient d'un corpus de démonstration, **vos nombres seront différents**. Trois faits doivent s'y retrouver, ceux du **slide 14**.

- Seule une très petite part des couples possibles a été vue, **le zéro est le cas normal**.
- Le comptage brut a la plus petite perplexité sur le train, et l'infini sur le dev. Il **récite** ce qu'il a compté, il ne prédit pas.
- Sur le dev, la perplexité descend puis remonte. La ligne `k retenu sur le dev` donne **votre** valeur de k.

En cas de problème

Si le script s'arrête sur `Introuvable`, le dossier **`seance3`** n'est pas à côté de **`seance2`**, ou **`merges.json`** manque. Revenez à l'étape 0.2.

Étape 5.2 — Le chiffre final sur le test

Étape 5.2. Lire la fin de l'écran (slide 15).

Vérification avant de continuer

```
perplexité sur le test, k = 0.01
  code           133.4
  français       327.7
  anglais        151.9
  darija         407.8
  hasard        2001.0   (un modèle qui tirerait au sort parmi |V| tokens)
```

Ici encore, vos nombres seront différents.

La ligne `hasard` est le repère. Un modèle qui ne saurait rien hésiterait entre les 2 001 tokens. Toute valeur plus petite mesure **ce que le modèle a appris du dépôt**.

Étape 5.3 — Vos propres chiffres

Étape 5.3. Lire vos chiffres (slides 14 et 15).

Répondez sur votre feuille, avec **vos** nombres.

1. Quelle valeur de k votre dev retient-il ? Quelle valeur le train aurait-il fait choisir, et pourquoi ce choix serait-il faux ?
2. Par combien votre modèle divise-t-il l'hésitation du hasard sur le code ?
3. Quelle langue a la perplexité la plus haute ? D'où vient cet écart, d'après le **slide 15** ?

Les parties du TP

1. Partie 0 — La mise en place
2. Partie 1 — Compter
3. Partie 2 — Prédire
4. Partie 3 — Lisser
5. Partie 4 — Mesurer
6. Partie 5 — Mesurer le dépôt
7. Partie 6 — Assainir
8. Partie 7 — Publier

Partie 6 — Assainir

Lien avec le cours

Cette partie applique le **slide 17** (L'artefact retient le corpus). L'étape 6.1 fait réciter le secret du cadre Problème, l'étape 6.2 écrit la fonction de la Définition, l'étape 6.3 vérifie que la table recomptée ne contient plus la clé.

Étape 6.1 — Le modèle récite un secret

Étape 6.1. Voir le modèle réciter (slide 17, Problème).

Relancez `python`. La fonction `decouper` coupe une ligne en mots et en signes.

```
>>> import re
>>> from bigramme import compter, predire
>>> def decouper(ligne):
...     return re.findall(r"\w+|\S", ligne)
...
>>> depot = ["for ticket in tickets", 'API_KEY = "sk-4f9a2c71"']
>>> decouper(depot[1])
['API_KEY', '=', '"', 'sk', '-', '4f9a2c71', '"']
```

Après la ligne `return`, appuyez sur Entrée une seconde fois. Restez dans l'interpréteur.

Étape 6.1 — Le modèle récite un secret (suite)

Le dépôt de deux lignes est compté tel quel, puis le modèle est interrogé au moment où sa réponse arrive sur le début de la clé.

```
>>> paires, contextes = compter([decouper(l) for l in depot])
>>> predire("sk", paires, contextes)
[('-', 1.0)]
>>> predire("-", paires, contextes)
[('4f9a2c71', 1.0)]
>>> exit()
```

Étape 6.1 — La lecture

- La ligne du secret n'apparaît **qu'une seule fois** dans le dépôt.
- Après `sk`, le modèle propose - avec la probabilité **1**, puis après - il propose `4f9a2c71` avec la probabilité **1**. **Une demande qui amène la réponse sur le début obtient la clé entière.**
- Le lissage n'y change rien, il garde les mêmes comptes, comme le montre le tableau du **slide 17**.

La seule protection est de retirer le secret **avant** de compter.

Étape 6.2 — La fonction masquer_secrets

Étape 6.2. Compléter la fonction masquer_secrets (slide 17, Définition).

Ouvrez **hygiene.py**. Remplacez la ligne `raise` par la ligne suivante.

```
    return AFFECTATION.sub(r"\1\2\3" + MASQUE + r"\3", ligne)
```

- `AFFECTATION`, défini au-dessus, reconnaît un nom qui annonce un secret (`KEY`, `SECRET`, `TOKEN`, `PASSWORD`, `MOT_DE_PASSE`), suivi de `=` ou de `:` et d'une valeur entre guillemets.
- `sub` réécrit ce qu'il a reconnu. `\1`, `\2` et `\3` recopient le nom, le signe et le guillemet, et `MASQUE` **remplace la valeur** par `<SECRET>`.

Enregistrez avec **Ctrl+S**.

Étape 6.2 — L'essai

Relancez `python`.

```
>>> from hygiene import masquer_secrets
>>> masquer_secrets('API_KEY = "sk-4f9a2c71"')
'API_KEY = "<SECRET>"'
>>> masquer_secrets("mot_de_passe: 'azerty123'")
"mot_de_passe: '<SECRET>'"
>>> masquer_secrets('titre = "Ticket ouvert"')
'titre = "Ticket ouvert"'
>>> exit()
```

Les deux secrets sont masqués, et la troisième ligne, qui n'en contient pas, **n'est pas modifiée**. Le script **`mesure_perplexite.py`** applique désormais cette fonction à chaque ligne avant de compter. Relancé, il n'affiche plus la ligne `hygiène pas encore écrite`.

Étape 6.3 — Tous les tests

Étape 6.3. Lancer tous les tests.

```
pytest -v
```

Vérification avant de continuer

Quatorze lignes se terminent par **PASSED**, et la dernière ligne est la suivante.

```
============================== 14 passed in 0.02s ==============================
```

Le dernier test, `test_p6_le_modele_recite_puis_ne_recite_plus`, compte deux fois le même dépôt, brut puis assaini, et vérifie que la clé **n'est plus dans la table**.

Partie 6 — Dans une autre application

Dans une autre application

Type d'application. Un hôpital dont les comptes rendus portent des noms de patients, une banque dont les dossiers portent des numéros de compte.
Comment. Le geste est le même, seul le motif change. Une expression régulière qui reconnaît un numéro de dossier ou un numéro de compte prend la place de `AFFECTATION`. La fonction passe **avant tout comptage, tout affinage et toute indexation**, car l'artefact produit retient ce qu'il a lu.

Les parties du TP

1. Partie 0 — La mise en place
2. Partie 1 — Compter
3. Partie 2 — Prédire
4. Partie 3 — Lisser
5. Partie 4 — Mesurer
6. Partie 5 — Mesurer le dépôt
7. Partie 6 — Assainir
8. Partie 7 — Publier

Partie 7 — Publier

Lien avec le cours

Cette partie applique le **slide 20** (Le contrat et la mesure sont acquis). Les trois commandes sont celles du cadre gris du **slide 20**.

Étapes 7.1 et 7.2 — Les fichiers à enregistrer

Étape 7.1. Regarder ce qui a changé.

Vous êtes toujours dans le dossier **seance3**.

```
git status
```

Git affiche en rouge, sous `Untracked files`, le dossier de la séance.

Étape 7.2. Choisir les fichiers à enregistrer.

La commande tient sur une seule ligne.

```
git add bigramme.py perplexite.py hygiene.py test_seance3.py mesure_perplexite.py
```

Git ne répond rien, c'est normal. Relancez `git status`, les cinq fichiers apparaissent maintenant en vert.

Étape 7.3 — Le *commit*

Étape 7.3. Enregistrer une version, le *commit*.

```
git commit -m "Séance 3, le modèle de langue et sa mesure, 14 tests verts"
```

Le texte entre guillemets est le message qui décrit la version. Git répond par une ligne qui commence par `[main` suivie de votre message, puis par le nombre de fichiers enregistrés.

En cas de problème

Si Git répond `nothing to commit`, l'étape 7.2 n'a pas été faite, ou a été lancée depuis un autre dossier que **`seance3`**.

Étape 7.4 — L'envoi sur GitHub

Étape 7.4. Envoyer la version sur GitHub.

```
git push
```

Vérification avant de continuer

La dernière ligne affichée contient `main -> main`. Ouvrez la page de votre dépôt **`iliacode`** sur GitHub et rafraîchissez-la. Le dossier **`seance3`** y apparaît, avec votre message de commit à côté.

Les deux fichiers durables, **`perplexite.py`** et **`hygiene.py`**, seront repris tels quels dans les séances suivantes.